

LAPORAN MODUL 14 PRAKTIKUM PARADIGMA PEMGROGRAMAN



Disusun oleh :

Nama : Irvan Cahya Nugraha
NIM : 245410034
Kelas : Informatika 1

**PROGRAM STUDI INFORMATIKA
PROGRAM SARJANA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS TEKNOLOGI DIGITAL INDONESIA
YOGYAKARTA
2024 / 2025**

MODUL 14

KONKURENSI DALAM JAVA

A. TUJUAN PRAKTIKUM

Tujuan dari modul praktikum ini adalah agar mahasiswa memahami konsep dasar konkurensi dalam Java

B. DASAR TEORI

Materi pada modul ini membahas konsep konkurensi dan thread dalam Java, pembuatan thread menggunakan Runnable, serta perilaku eksekusi thread yang berjalan secara bersamaan. Selanjutnya dipelajari race condition dan cara mengatasinya dengan sinkronisasi method menggunakan synchronized, termasuk penerapannya pada studi kasus perbankan. Modul ini juga membahas penggunaan ExecutorService dan thread pool sebagai solusi pengelolaan thread yang lebih efisien dibanding pembuatan thread manual.

C. PEMBAHASAN LISTING PRAKTIK

LATIHAN

D. PEMBAHASAN TUGAS

Tugas 1

```
class TugasCetak implements Runnable {
    private String nama;
    public TugasCetak(String nama) {
        this.nama = nama;
    }
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            // Thread.currentThread().getName() mengambil nama sistem thread
            System.out.println "[" + nama + "] mencetak angka: " + i);
            try {
                // Memberi jeda 500ms agar perpindahan thread terlihat jelas
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(nama + " terinterupsi.");
            }
        }
    }
}

public class PraktikumKonkurensi {
    public static void main(String[] args) {
        System.out.println("=== Memulai Program Multi-threading ===");
        // 1. Membuat objek Runnable
        TugasCetak tugasA = new TugasCetak("Thread A");
        TugasCetak tugasB = new TugasCetak("Thread B");
        // 2. Membuat objek Thread asli dari Java
        Thread t1 = new Thread(tugasA);
        Thread t2 = new Thread(tugasB);
        // 3. Menjalankan Thread
        t1.start();
        t2.start();
        try {
            // 4. Menunggu t1 dan t2 selesai sebelum lanjut ke baris bawahnya
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("=== Semua Thread Selesai, Main Program Keluar ===");
    }
}
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE  COMMENTS

=== Memulai Program Multi-threading ===
[Thread A] mencetak angka: 1
[Thread B] mencetak angka: 1
[Thread A] mencetak angka: 2
[Thread B] mencetak angka: 2
[Thread B] mencetak angka: 3
○ [Thread A] mencetak angka: 3
[Thread B] mencetak angka: 4
[Thread A] mencetak angka: 4
[Thread B] mencetak angka: 5
[Thread A] mencetak angka: 5
=== Semua Thread Selesai, Main Program Keluar ===
PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\14>
```

Pembahasan : Pada praktik ini, program menjalankan dua thread (Thread A dan Thread B) secara bersamaan menggunakan interface Runnable. Hasil output angka muncul selang-seling karena kedua thread dieksekusi secara konkurensi oleh JVM, bukan secara berurutan. Penggunaan Thread.sleep() membuat perpindahan eksekusi antar thread terlihat jelas, sedangkan method join() memastikan program utama menunggu hingga seluruh thread selesai sebelum menampilkan pesan akhir.

Tugas 2

Welcome

Snapcode

PraktikumKonkurensi.java

Praktikum2.java

Praktikum2.java > ...

1class Counter {

2int count = 0;

3

4// Gunakan kata kunci 'synchronized' untuk memperbaiki

5public synchronized void increment () {

6count++;

7}

8}

9

10public class Praktikum2 {

11Run | Debug

12public static void main(String[] args) throws InterruptedException {

13Counter c = new Counter();

14

15Runnable task = () -> {

16for (int i = 0; i < 1000; i++) {

17c.increment();

18}

19};

20Thread t1 = new Thread(task);

21Thread t2 = new Thread(task);

22

23t1.start(); t2.start();

24t1.join(); t2.join(); // Menunggu kedua thread selesai

25

26System.out.println("Hasil Akhir Counter: " + c.count);

27}

28}

29

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

AZURE

COMMENTS

PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\14> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetails' '-Duser.dir=D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\14' '-jar' 'Praktikum2.jar'

Hasil Akhir Counter: 1807

PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\14>

```

Welcome | Snapcode | J PraktikumKonkurensi.java | J Praktikum2.java X
J Praktikum2.java > Counter > increment()
1 class Counter {
2     int count = 0;
3
4     // Gunakan kata kunci 'synchronized' untuk memperbaiki
5     public void increment () {
6         count++;
7     }
8 }
9
10 public class Praktikum2 {
11     Run | Debug
12     public static void main(String[] args) throws InterruptedException {
13         Counter c = new Counter();
14
15         Runnable task = () -> {
16             for (int i = 0; i < 1000; i++) {
17                 c.increment();
18             }
19         };
20
21         Thread t1 = new Thread(task);
22         Thread t2 = new Thread(task);
23
24         t1.start(); t2.start();
25         t1.join(); t2.join(); // Menunggu kedua thread selesai
26
27         System.out.println("Hasil Akhir Counter: " + c.count);
28     }
29 }

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS AZURE COMMENTS

```

Data\Roaming\Code\User\workspaceStorage\e2bd0d7492798f882ac6add49bde5dfc\redhat.java\jdt_ws\14_6ab36d77\bin' 'Praktikum2'
Hasil Akhir Counter: 1807
PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\14> ^C
PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\14> d;; cd 'd:\AREAS\Semester3\Praktikum Paradigma Pemrograman\14'; & 'C:\Program Files\Java\jdk-11.0.2\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\ASUS\AppData\Roaming\Code\User\workspaceStorage\e2bd0d7492798f882ac6add49bde5dfc\redhat.java\jdt_ws\14_6ab36d77\bin' 'Praktikum2'
Hasil Akhir Counter: 2000
PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\14> d;; cd 'd:\AREAS\Semester3\Praktikum Paradigma Pemrograman\14'; & 'C:\Program Files\Java\jdk-11.0.2\bin\java.exe' -XX:+ShowCodeDetailsInExceptionMessages -cp 'C:\Users\ASUS\AppData\Roaming\Code\User\workspaceStorage\e2bd0d7492798f882ac6add49bde5dfc\redhat.java\jdt_ws\14_6ab36d77\bin' 'Praktikum2'
Ready
Ln 5, Col 1

```

Pembahasan : Pada praktik ini, dua thread menambah nilai variabel count secara bersamaan melalui method increment(). Tanpa kata kunci synchronized, terjadi race condition karena proses count++ tidak dieksekusi secara atomik, sehingga hasil akhir bisa kurang dari nilai yang diharapkan. Sinkronisasi diperlukan agar hanya satu thread yang dapat mengakses method tersebut pada satu waktu sehingga data tetap konsisten.

Tugas 3

```
Welcome  Snapcode  J PraktikumKonkurensiJava  J Praktikum3.java  J Praktikum2.java

J Praktikum3.java > Praktikum3 > main(String[])
1  import java.util.concurrent.ExecutorService;
2  import java.util.concurrent.Executors;
3
4  public class Praktikum3 {
    Run | Debug
5      public static void main(String[] args) {
6          // Membuat pool dengan 3 pekerja (thread)
7          ExecutorService executor = Executors.newFixedThreadPool(nThreads: 3);
8
9          for (int i = 1; i <= 10; i++) {
10             int idTugas = i;
11             executor.execute(() -> {
12                 System.out.println("Tugas " + idTugas + " dikerjakan oleh " + Thread.currentThread().getName());
13                 try { Thread.sleep(millis: 1000); } catch (Exception e) {}
14             });
15         }
16         executor.shutdown(); // Berhenti menerima tugas baru
17     }
18 }
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  AZURE  COMMENTS

Tugas 4 dikerjakan oleh pool-1-thread-2
Tugas 5 dikerjakan oleh pool-1-thread-1
Tugas 6 dikerjakan oleh pool-1-thread-3
Tugas 9 dikerjakan oleh pool-1-thread-1
Tugas 7 dikerjakan oleh pool-1-thread-3
Tugas 8 dikerjakan oleh pool-1-thread-2
Tugas 10 dikerjakan oleh pool-1-thread-2
○ Tugas 4 dikerjakan oleh pool-1-thread-2
Tugas 5 dikerjakan oleh pool-1-thread-1
Tugas 6 dikerjakan oleh pool-1-thread-3
Tugas 9 dikerjakan oleh pool-1-thread-1
Tugas 7 dikerjakan oleh pool-1-thread-3
Tugas 8 dikerjakan oleh pool-1-thread-2
Tugas 4 dikerjakan oleh pool-1-thread-2
Tugas 5 dikerjakan oleh pool-1-thread-1
Tugas 6 dikerjakan oleh pool-1-thread-3
Tugas 4 dikerjakan oleh pool-1-thread-2
Tugas 5 dikerjakan oleh pool-1-thread-1
Tugas 4 dikerjakan oleh pool-1-thread-2
Tugas 4 dikerjakan oleh pool-1-thread-2
Tugas 5 dikerjakan oleh pool-1-thread-1
Tugas 6 dikerjakan oleh pool-1-thread-3
Tugas 4 dikerjakan oleh pool-1-thread-2
Tugas 5 dikerjakan oleh pool-1-thread-1
Tugas 4 dikerjakan oleh pool-1-thread-2
Tugas 5 dikerjakan oleh pool-1-thread-1
Tugas 6 dikerjakan oleh pool-1-thread-3
Tugas 9 dikerjakan oleh pool-1-thread-1
Tugas 4 dikerjakan oleh pool-1-thread-2
Tugas 5 dikerjakan oleh pool-1-thread-1
Tugas 6 dikerjakan oleh pool-1-thread-3
Tugas 4 dikerjakan oleh pool-1-thread-2
Tugas 4 dikerjakan oleh pool-1-thread-2
Tugas 5 dikerjakan oleh pool-1-thread-1
Tugas 6 dikerjakan oleh pool-1-thread-3
Tugas 9 dikerjakan oleh pool-1-thread-1
Tugas 7 dikerjakan oleh pool-1-thread-3
Tugas 8 dikerjakan oleh pool-1-thread-2
Tugas 10 dikerjakan oleh pool-1-thread-2
PS D:\AREAS\Semester3\Praktikum Paradigma Pemrograman\14>
```

Pembahasan : Pada praktik ini digunakan ExecutorService dengan FixedThreadPool berjumlah tiga thread untuk mengeksekusi sepuluh tugas. Output menunjukkan bahwa tugas-tugas dikerjakan bergantian oleh thread yang sama, menandakan thread digunakan kembali. Pendekatan ini lebih efisien dibanding membuat thread manual karena menghemat sumber daya dan memudahkan pengelolaan konkurensi.

Tugas 4

```
class RekeningBank {
    private int saldo = 1000000;

    // Method disinkronisasi agar hanya 1 orang yang bisa akses di satu waktu
    public synchronized void tarikUang(String nama, int jumlah) {
        System.out.println(nama + " mencoba menarik: Rp " + jumlah);

        if (saldo >= jumlah) {
            System.out.println(nama + " berhasil melakukan pengecekan saldo.");
            try {
                // Simulasi proses bank yang membutuhkan waktu
                Thread.sleep(100);
            } catch (InterruptedException e) {}

            saldo -= jumlah;
            System.out.println(nama + " transaksi SUKSES. Sisa saldo: Rp " + saldo);
        } else {
            System.out.println(nama + " transaksi GAGAL. Saldo tidak cukup! Sisa: Rp " + saldo);
        }
        System.out.println("-----");
    }
}

public class KasusBank {
    public static void main(String[] args) {
        RekeningBank akunBersama = new RekeningBank();

        // Membuat dua thread (Suami dan Istri) yang mengakses objek yang sama
        Thread suami = new Thread(() -> akunBersama.tarikUang("Suami", 700000));
        Thread istri = new Thread(() -> akunBersama.tarikUang("Istri", 700000));

        suami.start();
        istri.start();
    }
}
```

Pembahasan : Pada studi kasus rekening bank, method tarikUang() diberi kata kunci synchronized untuk mencegah dua thread menarik saldo secara bersamaan. Dengan sinkronisasi, pengecekan saldo dan pengurangan saldo dilakukan secara aman sehingga hanya satu transaksi yang berhasil jika saldo tidak mencukupi. Praktik ini menunjukkan penerapan sinkronisasi dalam kasus nyata untuk mencegah kesalahan data finansial.

E. KESIMPULAN

Modul praktikum ini menunjukkan bahwa konkurensi memungkinkan program Java berjalan lebih efisien dengan banyak thread, namun berisiko menimbulkan race condition jika tidak dikelola dengan benar. Penggunaan synchronized sangat penting untuk menjaga konsistensi data bersama, sedangkan ExecutorService memberikan solusi manajemen thread yang lebih modern, aman, dan efisien dibanding pembuatan thread secara manual.

F. LAMPIRAN

